



**TECHNICAL UNIVERSITY OF MUNICH**

**SCHOOL OF COMPUTATION, INFORMATION AND  
TECHNOLOGY  
INFORMATICS**

Master's Thesis in Informatics

**DRAFT - Thesis Proposal:  
Something With Usability in PIX:E**

Rai Trouvain



**TECHNICAL UNIVERSITY OF MUNICH**

**SCHOOL OF COMPUTATION, INFORMATION AND  
TECHNOLOGY  
INFORMATICS**

Master's Thesis in Informatics

**DRAFT - Thesis Proposal:**  
**Something With Usability in PIX:E**  
Thesis Proposal: Irgendetwas mit Usability  
in PIX:E

|                         |                      |
|-------------------------|----------------------|
| <b>Author:</b>          | Rai Trouvain         |
| <b>Supervisor:</b>      | Prof. Dr. Georg Groh |
| <b>Advisors:</b>        | Julian Geheeb M.Sc.  |
| <b>Submission Date:</b> | DRAFT - 13.06.2026   |



## Disclaimer

I confirm that this Master's thesis is my own work and I have documented all sources and material used.

---

(Date)

---

(Rai Trouvain)



# Acknowledgements

The acknowledgements

## **Abstract**

English Abstract

## **Zusammenfassung**

German Abstract



# Contents

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>1 Introduction .....</b>                         | <b>2</b>  |
| <b>2 Background .....</b>                           | <b>3</b>  |
| <b>3 Path-Based Diagrams .....</b>                  | <b>4</b>  |
| 3.1 Path Selection .....                            | 4         |
| 3.1.1 Functionality .....                           | 4         |
| 3.1.2 Implementation .....                          | 4         |
| 3.1.3 Workflow .....                                | 4         |
| 3.2 Diagram Generation .....                        | 5         |
| 3.2.1 Functionality .....                           | 5         |
| 3.2.2 Implementation .....                          | 6         |
| 3.2.3 Workflow .....                                | 6         |
| 3.2.4 Limitations .....                             | 6         |
| <b>4 Locks and Keys .....</b>                       | <b>7</b>  |
| 4.1 Creating and Visualizing Locks and Keys .....   | 7         |
| 4.2 Pathfinding with Locks and Keys .....           | 7         |
| 4.2.1 Basic Dijkstra with Locks and Keys .....      | 7         |
| 4.2.2 Fixed Keys .....                              | 8         |
| 4.2.3 Soft Gates .....                              | 8         |
| 4.2.4 Consumable Keys and Soft-Lock Detection ..... | 9         |
| 4.3 Bidirectional Edges and Backtracking .....      | 11        |
| 4.3.1 Example .....                                 | 11        |
| 4.3.2 Full Algorithm .....                          | 11        |
| 4.4 Options/Settings .....                          | 11        |
| 4.5 Testing .....                                   | 11        |
| 4.6 Limitations and Future Workflow .....           | 11        |
| <b>Bibliography .....</b>                           | <b>12</b> |

# 1 Introduction

## 2 Background

## 3 Path-Based Diagrams

- basic idea already present in PaceMaker: select path in statechart and visualize data along path
- main design guideline: build upon high configurability already present in the system
- following sections describe functionalities, implementation choices and workflow/usage of these features implemented in pixe

### 3.1 Path Selection

#### 3.1.1 Functionality

- selection of nodes in statechart triggers calculation of path connecting them
- selected path is highlighted (visual feedback)
- path is subsequently used for diagram generation

#### 3.1.2 Implementation

- select start + end node, optionally intermediate -> calculate path using dijkstra
- `pathsApi.ts`: to allow for different algorithms in the future (e.g. specific pathfinding for nestedness and concurrency)

#### 3.1.3 Workflow

- selection of at least two nodes
  - Ctrl + click
- path is calculated along nodes **in order of selection**
- if path is found, the corresponding nodes are highlighted
- path can be de-selected by removing node selection (clicking anywhere)

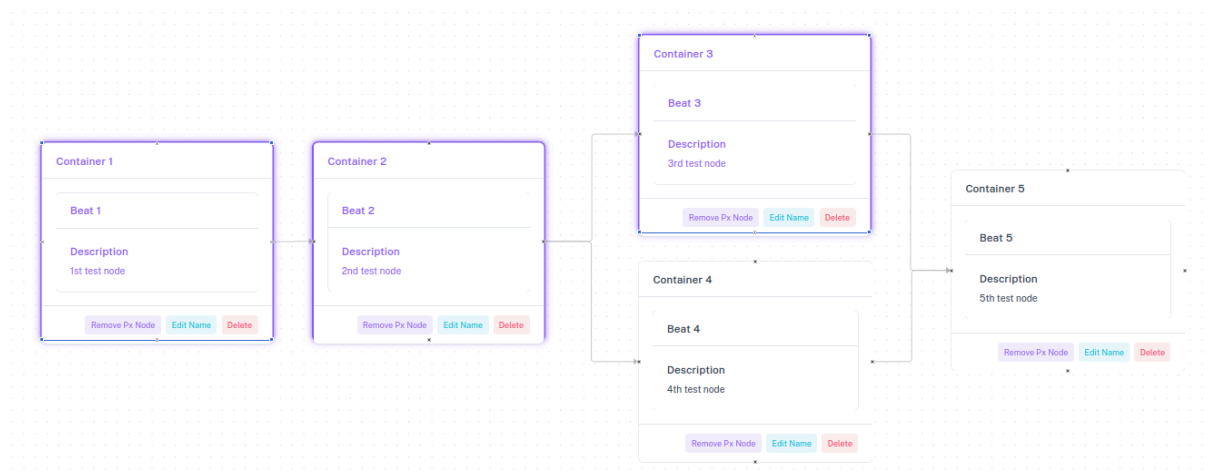


Figure 1: Path highlighted in state chart

## 3.2 Diagram Generation

### 3.2.1 Functionality

- expandable element on statechart page
  - usability: users can hide diagrams while working on statechart
- can create multiple diagrams and remove diagrams (red because diagrams do not persist)

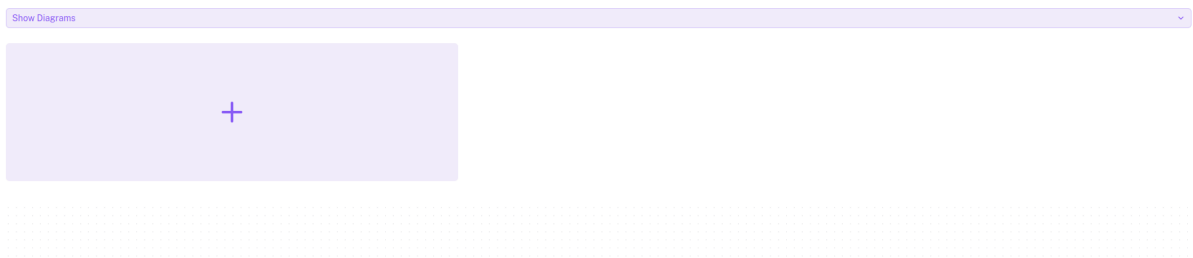


Figure 2: Upper section of the charts page with expandable element and UI for diagram creation

- selection of one or multiple component definitions for the y axis
- optionally selection of one component for the x axis (e.g. estimated playtime)
  - values are summed up along selected path
  - default: equal spacing

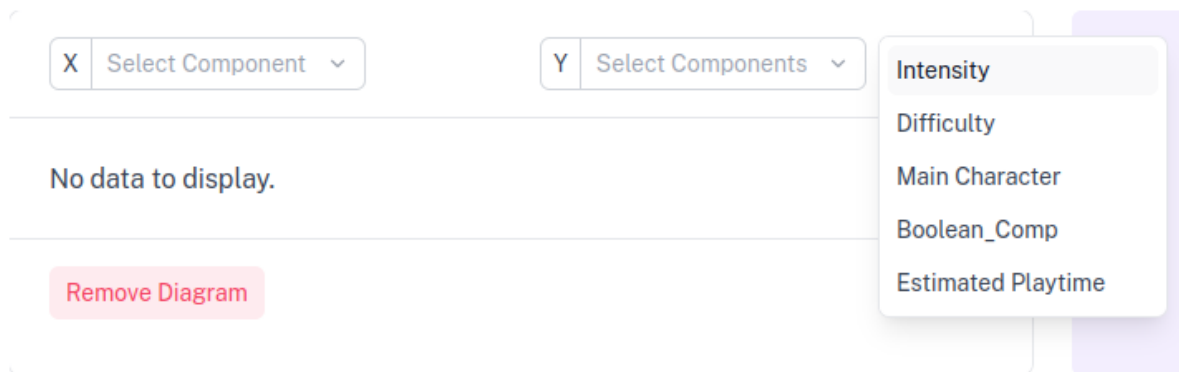


Figure 3: Selectors for components to be mapped to X/Y axes

- selected path along x axis
  - if no path selected: all nodes,  sorted



Figure 4: Diagram visualizing component along selected path

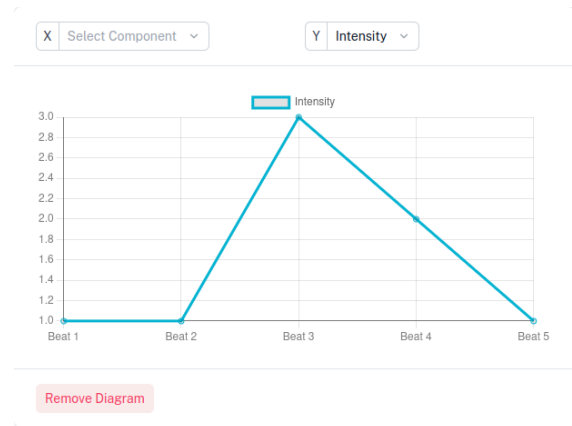


Figure 5: Diagram visualizing component across all nodes

### 3.2.2 Implementation

- [chartJS](#) library
- modular architecture: `PxDiagrams` as wrapper element
  - manages diagrams
    - currently: passes path (reactively), deletes diagrams
    - future work: path snapshots, different data types
- data is extracted from nodes once per diagram, switching between axis configurations is then done via different parsing configurations

### 3.2.3 Workflow

1. add new diagram
2. in any order:
  - select one or no component for x axis
  - select one or more component for y axis
  - select path
- changes in any of the configurations will be reflected live in the diagram

### 3.2.4 Limitations

- diagrams do not persist (reset when reloading page)
- only line diagrams for now
- same path selection for all diagrams

## 4 Locks and Keys

requirements/context:

- multiple keys possible per node
- multiple locks possible per node
- pathfinding and reachability analysis

### 4.1 Creating and Visualizing Locks and Keys

### 4.2 Pathfinding with Locks and Keys

#### 4.2.1 Basic Dijkstra with Locks and Keys

- goal: all locks assigned to edges in the path have to be unlocked by key(s) collected up to the edge in question
- inventory: track keys collected along path to determine whether encountered edges can be unlocked
  - inventory similar to ([Aversa, Sardina, and Vassos 2015](#))
  - modeled via `PxKeySet` -> formally: mapping of key defs to counts
- basic form: inventory associated with a node reflects keys collected along shortest path to this node and keys made available in this node -> later extended (consumable keys)

##### 4.2.1.1 Unlocking Function

- `canUnlock()`: given (multi-)sets of locks and keys, determine whether locks can be unlocked
  - added complexity due to an edge potentially having multiple locks, and each locks potentially being unlocked by multiple keys
  - exponential blowup, but no optimization so far as average case is expected to be less complex

given set of keys in inventory  $K$  and locks on edge  $L$ :

```
canUnlock(K, L):
  if L = ∅
    return TRUE

  unlockingKeysets = required(l1) × ... × required(ln)

  if ∃ U ∈ unlockingKeysets where ∀ u ∈ U. u ∈ K:
    return TRUE
```

```

else
    return FALSE

```

```

1 canUnlock(K, L):
2   if  $L = \emptyset$ 
3     | return TRUE
4   unlockingKeysets = required(l1)  $\times$  ...  $\times$  required(ln)
5   if  $\exists U \in \text{unlockingKeysets}$  where  $\forall u \in U. u \in K$ :
6     | return TRUE
7   else
8     | return FALSE

```

#### 4.2.1.2 Example

![[can-unlock-without-consume.canvas]]

- $I = \{X, Y\} \rightarrow \text{false}$
- $I = X, Y, Z \rightarrow \text{true}$

#### 4.2.1.3 Full Algorithm

![[basic-lock-key-pathfinding.canvas|800]]

#### 4.2.1.4 Example Iteration

- TODO add small example of iteration

### 4.2.2 Fixed Keys

- goal: fixed keys can only unlock locks on outgoing edges of the node they are assigned to

#### implementation:

- do not add keys to inventory of neighbors
-  add example

### 4.2.3 Soft Gates

- goal: soft gates **can** be passed without the corresponding key, but this should be indicated in the path highlighting

#### implementation:

- during pathfinding, ignore soft-gate locks
- after path has been found, re-trace it to determine whether keys are available or not
-  add example from pixe



#### 4.2.4 Consumable Keys and Soft-Lock Detection

- goal: inventory should reflect usage of single-use keys
- variants of inventory -> idea of ‘parallel universes’
  - nodes exist in different realities if they have different inventories
  - assign each node a set of possible keysets

##### implementation:

- maintain set of `PxKeySets` per node
- in `canUnlock`, consider multiplicity of consumable keys
- for updating inventory of neighbor nodes: remove consumed keys from inventory
  - if there are multiple possible constellations, add multiple keysets

##### 4.2.4.1 Pathfinding with Consumable Keys

###### 4.2.4.1.1 Extending Unlocking Check

given multiset of keys in inventory  $K$  and locks on edge  $L$ :

```

title="Unlocking function"
canUnlock(K, L):
  if L = ∅
    return TRUE

  unlockingKeysets = required(l1) × ... × required(ln)

  if ∃ U ∈ unlockingKeysets where U ⊆ K:
    return TRUE
  else
    return FALSE

```

- note: subset of multiset

![[can-unlock-with-consume.canvas]]

- now:  $I = \{X, Z\} \rightarrow \text{false}$

###### 4.2.4.1.2 Removing Consumable Keys from Inventory

given inventory  $I$  containing multisets of keys  $K$  and locks on edge  $L$ :

```

title="Removing consumed keys"
canUnlock(I, L):
  if L = ∅
    return TRUE

  consumableRequirements = locks with at least one consumable key unlocking

```

```

unlockingKeysets = required(l1)  $\times$  ...  $\times$  required(ln)

updatedInventory = {}
for K_i  $\in$  I:
    if  $\exists U \in$  unlockingKeysets with  $K_i \subseteq U$ :
        updatedInventory.push( $K_i - \{ \text{key} \in U. \text{consumable}(\text{key}) \}$ )

return updatedInventory

```

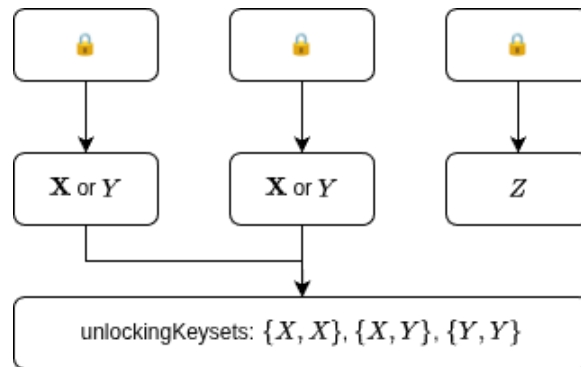


Figure 6: Calculation of unlocking keysets for key consumption

- $I = \{X, X, Z\} \rightarrow \{Z\}$

#### 4.2.4.1.3 Example

-  example in pixe for consumable keys

#### 4.2.4.2 Soft-Locks

[(Mawhorter and Smith 2021)]: >it is possible to reach the goal state from every reachable state

- here: not nodes, but **states in which nodes are visited**

![[soft-lock-example.canvas|200]]

##### implementation:

- potential soft-lock state: node unlockable with some, but not all keysets
- when removing consumed keys: remove empty keysets entirely. then, compare inventory length before and after consumption

#### 4.2.4.2.1 Example

-  example in pixe for consumable keys

### 4.3 Bidirectional Edges and Backtracking

- goal: allow for longer/cyclic paths needed to collect keys
- another idea of parallel universes
-  add citation from julians paper?

#### implementation:

- alternative used here: nodes are re-added to queue iff they have a different inventory assigned to them than in previous encounters -> only duplicate nodes as needed
  - for each node, maintain inventories it has been visited with
    - only re-visit with a different inventory
  - add limiting constant for revisits
  - also maintain unlocked edges

![[backtracking.canvas]]

- critical path:  $A - B - A - C$

![[lock-key-pathfinding-backtracking.canvas|800]]

#### 4.3.1 Example

-  example screenshot from pixe

#### 4.3.2 Full Algorithm

-  make diagram of full algorithm?

### 4.4 Options/Settings

-  describe pathfinding options

### 4.5 Testing

-  describe

### 4.6 Limitations and Future Workflow

- difficulty estimation
- recommendations for lock/key types ([Dormans 2017](#))
- LLM-based consistency check with descriptions

## Bibliography

Davide Aversa, Sebastian Sardina, and Stavros Vassos. 2015. Path Planning with Inventory-Driven Jump-Point-Search. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 11(1):2–8.

Joris Dormans. 2017. Cyclic Generation.

Ross Mawhorter and Adam Smith. 2021. Softlock Detection for Super Metroid with Computation Tree Logic. In *Proceedings of the 16th International Conference on the Foundations of Digital Games*, pages 1–10, New York, NY, USA. Association for Computing Machinery.